

Informe Técnico

Ruta óptima entre ciudades mediante grafos ponderados

Nombres: Ankatu Alcaman, Fabián Neira, Francisco Martinez, Jenniffer Urrutia

Asignatura: Matemáticas Discretas

Docente: Elliott Jamil Mardones Arias

Fecha: 22/06/2026

1. Introducción

El presente informe detalla el diseño e implementación de una solución computacional basada en la teoría de grafos para resolver el problema del camino mínimo. El objetivo principal es modelar una red interurbana y calcular la ruta más eficiente entre una ciudad de origen y una de destino. Para este proyecto, se seleccionó un conjunto de 15 ciudades pertenecientes a Colombia, estableciendo un mínimo de 20 conexiones reales entre ellas. El criterio de ponderación (peso) asignado a cada arista corresponde a la distancia estimada en kilómetros

entre las ciudades conectadas, manteniendo este parámetro de forma consistente para evaluar el costo de los recorridos.

2. Modelamiento Matemático

El sistema interurbano se representó mediante un grafo ponderado no dirigido, definido formalmente como $G = (V, E, w)$

- **Vértices (V):** Corresponde al conjunto finito de 15 ciudades colombianas seleccionadas. $V = \{\text{Bogotá, Medellín, Cali, Barranquilla, Cartagena, Bucaramanga, Cúcuta, Pereira, Manizales, Santa Marta, Villavicencio, Ibagué, Pasto, Montería, Sincelejo}\}$
- **Aristas (E):** Representa el conjunto de 22 conexiones viales reales habilitadas entre los nodos del grafo. $E = \{(u,v) \mid \text{existe conectividad directa entre la ciudad } u \text{ y la ciudad } v\}$
- **Función de peso (w):** Es una función $w: E \rightarrow \mathbb{R}^+$ (reales positivos) que asocia a cada arista (u,v) perteneciente a E un valor correspondiente a la distancia en kilómetros. Por ejemplo, $w(\text{Bogotá, Medellín}) = 415$.

El problema se define dado un nodo de origen "s" perteneciente a V y un destino "t" perteneciente a V , donde se busca un camino $P = (s = v_0, v_1, v_2, \dots, v_k = t)$. El costo total del camino $C(P)$ está dado por la suma de sus pesos: $C(P) = \text{Suma de } w(v_i, v_{i+1}) \text{ desde } i=0 \text{ hasta } k-1$

La ruta óptima es el camino P^* que minimiza dicho costo entre todas las trayectorias posibles en el grafo conexo: $P^* = \arg \min C(P)$

3. Metodología: Algoritmo de Camino Mínimo

Para la resolución de este problema, se seleccionó e implementó el Algoritmo de **Dijkstra**. La elección se fundamenta en las propiedades topológicas del grafo modelado: al representar distancias físicas entre ciudades, la función de peso "w" siempre devuelve valores estrictamente positivos ($w > 0$). **Dijkstra** es el algoritmo idóneo, ya que garantiza encontrar el camino mínimo de fuente única en grafos sin aristas de peso negativo, operando mediante la relajación sucesiva de los nodos más cercanos.

Comparación con alternativa: Una alternativa matemática posible es el algoritmo de Bellman-Ford. bien Bellman-Ford cumple el mismo objetivo, está diseñado para tolerar grafos con pesos negativos. Esto conlleva una complejidad temporal superior de $O(|V||E|)$. Dado que la red de ciudades implementada carece de distancias negativas, el uso de Dijkstra apoyado por una cola de prioridad reduce drástico el costo computacional a $O((|V| + |E|) \log |V|)$, resultando significativamente más eficiente y pertinente para el contexto del proyecto.

4. Implementación

La solución se desarrolló en Python, siguiendo un paradigma modular que separa la lógica del grafo (cálculo), la optimización algorítmica y la visualización de datos. El sistema integra cuatro librerías clave que aseguran eficiencia y escalabilidad.

4.1 Componentes Técnicos:

- **NetworkX:** Utilizada como motor para la estructura de datos abstracta. Permite definir la red interurbana como un grafo no dirigido (`nx.Graph()`), donde los nodos representan ciudades y las aristas almacenan la propiedad `weight` (distancia en km). Esta librería provee acceso de complejidad constante $O(1)$ para la consulta de adyacencias.
- **heapq:** Esencial para la optimización del algoritmo de Dijkstra. La librería implementa internamente un *Min-Heap* (montículo binario) sobre una lista convencional. Esto garantiza que la operación de extracción del nodo con menor distancia acumulada se realice en tiempo logarítmico $O(\log |V|)$, optimizando drásticamente el rendimiento en comparación con una búsqueda lineal.
- **Tkinter:** Gestión de la Interfaz Gráfica de Usuario (GUI). Permite una interacción intuitiva mediante widgets `ttk.Combobox` para la selección de ciudades origen/destino, asegurando la integridad de los datos mediante estados de solo lectura.
- **Matplotlib:** Renderizado gráfico del grafo. Se implementa en dos capas: una base para la topología total de la red y una capa superior que grafica exclusivamente el camino óptimo P^* con un grosor de trazo incrementado (`width=3.5`) y color diferenciado (`crimson`), facilitando la interpretación visual de la solución.

4.2 Fragmento de lógica central (Dijkstra):

El proceso de relajación de aristas, núcleo de la optimización del camino, se implementa mediante la siguiente estructura de control:

Python

```
# Lógica de relajación para encontrar el camino mínimo
```

```
if nueva_distancia < distancias[vecino]:
```

```
    distancias[vecino] = nueva_distancia
```

```
    predecesores[vecino] = nodo_actual
```

```
    heapq.heappush(cola_prioridad, (nueva_distancia, vecino))
```

5. Resultados y Pruebas

A continuación, se presentan tres escenarios de prueba ejecutados exitosamente en la aplicación desarrollada:

Prueba	Origen	Destino	Ruta Óptima	Costo (km)
Prueba 1	Bogotá	Cali	Bogotá → Ibagué → Cali	470
Prueba 2	Cúcuta	Manizales	Cúcuta → Bucaramanga → Medellín → Manizales	820
Prueba 3	Cartagena	Bogotá	Cartagena → Sincelejo → Medellín → Bogotá	1045

6. Conclusiones

El desarrollo del proyecto demostró empíricamente cómo los conceptos teóricos de matemática discreta son directamente aplicables a la resolución de problemas de logística e infraestructura. La dificultad técnica principal radicó en la sincronización del algoritmo matemático con la librería de visualización gráfica, evitando bloqueos en la interfaz mediante el correcto manejo de las figuras. Como mejora futura, el modelo matemático podría extenderse incorporando grafos dirigidos para simular sentidos vehiculares, o la integración de múltiples variables de peso (como tráfico en tiempo real) que dinamicen el cálculo del camino mínimo.

7. Repositorio en GitHub

El código fuente completo, los datos de configuración del grafo, el archivo requirements.txt y las instrucciones de ejecución (README.md) se encuentran alojados y disponibles públicamente en el siguiente enlace:

Enlace del repositorio:

https://github.com/fmartinez2026-lab/proyecto-mate.discreta/blob/main/proyecto_%20mate_discreta.py